

MISSÃO 1 - O Retângulo do Caos

Crie um retângulo com largura e altura fornecidas pelo usuário. Ao pressionar a barra de espaço, gere uma nova combinação aleatória de cores para o retângulo e o fundo da janela.

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
import random

# ===== Classe do Retângulo ===== #
class Rectangle:

    def __init__(self, width, height):

        self.width = width
        self.height = height

        self.color = self.random_color()

        # ===== Shared Vertex ===== #
        self.vertices = [
            (-width / 2, height / 2), # v0
            ( width / 2, height / 2), # v1
            ( width / 2, -height / 2), # v2
            (-width / 2, -height / 2) # v3
        ]

        self.indices = [
            0, 1, 2,
            2, 3, 0
        ]

        # ===== Cor aleatória ===== #
        def random_color(self):
            return (
                random.random(),
                random.random(),
                random.random()
            )

        # ===== Atualizar cor ===== #
        def update_color(self):
            self.color = self.random_color()

        # ===== Desenhar ===== #
        def draw(self):

            glColor3f(*self.color)

            glBegin(GL_TRIANGLES)

            for index in self.indices:
                glVertex2f(*self.vertices[index])

            glEnd()

# ===== Variáveis globais ===== #
rect = None

bg_color = (0.0, 0.0, 0.0)

# ===== Display ===== #
def display():

    glClearColor(*bg_color, 1.0)

    glClear(GL_COLOR_BUFFER_BIT)

    rect.draw()

    glutSwapBuffers()

# ===== Teclado ===== #
```

```

def keyboard(key, x, y):

    global bg_color

    # Barra de espaço
    if key == b' ':

        rect.update_color()

        bg_color = (
            random.random(),
            random.random(),
            random.random()
        )

        glutPostRedisplay()

# ===== Inicialização ===== #
def init():

    glMatrixMode(GL_PROJECTION)

    glLoadIdentity()

    gluOrtho2D(-100, 100, -100, 100)

# ===== Programa principal ===== #
def main():

    global rect

    width = float(input("Digite a largura do retângulo: "))
    height = float(input("Digite a altura do retângulo: "))

    rect = Rectangle(width, height)

    glutInit()

    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)

    glutInitWindowSize(600, 600)

    glutCreateWindow(b"Retangulo do Caos")

    init()

    glutDisplayFunc(display)

    glutKeyboardFunc(keyboard)

    glutMainLoop()

if __name__ == "__main__":
    main()

```

✓ Missão 2: O Pannel das Formas

Crie uma cena com “quatro formas geométricas”. Cada forma (círculo, quadrado, triângulo, e outra que você deve escolher) deve ser desenhada em uma área própria da janela. Utilize cores variadas para cada forma.

```

from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

import math
import random

# ===== Classe Base ===== #
class Shape:

    def __init__(self, x, y, color):

        self.x = x
        self.y = y
        self.color = color

```

```

        self.vertices = []
        self.indices = []

    def draw(self):

        glColor3f(*self.color)

        glBegin(GL_TRIANGLES)

        for index in self.indices:
            vx, vy = self.vertices[index]

            glVertex2f(vx + self.x, vy + self.y)

        glEnd()

# ===== Quadrado ===== #
class Square(Shape):

    def __init__(self, x, y, size, color):

        super().__init__(x, y, color)

        s = size / 2

        # Shared vertex
        self.vertices = [
            (-s, s), # v0
            (s, s), # v1
            (s, -s), # v2
            (-s, -s) # v3
        ]

        self.indices = [
            0, 1, 2,
            2, 3, 0
        ]

# ===== Triângulo ===== #
class Triangle(Shape):

    def __init__(self, x, y, size, color):

        super().__init__(x, y, color)

        s = size

        self.vertices = [
            (0, s),
            (-s, -s),
            (s, -s)
        ]

        self.indices = [
            0, 1, 2
        ]

# ===== Hexágono ===== #
class Hexagon(Shape):

    def __init__(self, x, y, radius, color):

        super().__init__(x, y, color)

        # Centro
        self.vertices.append((0, 0))

        # Vértices externos
        for i in range(6):

            angle = math.radians(i * 60)

            vx = radius * math.cos(angle)
            vy = radius * math.sin(angle)

            self.vertices.append((vx, vy))

# Triângulos

```

```

    for i in range(1, 7):

        next_i = i + 1 if i < 6 else 1

        self.indices.extend([
            0, i, next_i
        ])

# ===== Círculo ===== #
class Circle(Shape):

    def __init__(self, x, y, radius, color, segments=30):

        super().__init__(x, y, color)

        # Centro
        self.vertices.append((0, 0))

        # Circunferência
        for i in range(segments):

            angle = 2 * math.pi * i / segments

            vx = radius * math.cos(angle)
            vy = radius * math.sin(angle)

            self.vertices.append((vx, vy))

        # Índices
        for i in range(1, segments):

            self.indices.extend([
                0, i, i + 1
            ])

        self.indices.extend([
            0, segments, 1
        ])

# ===== Lista de formas ===== #
shapes = []

# ===== Display ===== #
def display():

    glClear(GL_COLOR_BUFFER_BIT)

    for shape in shapes:
        shape.draw()

    glutSwapBuffers()

# ===== Inicialização ===== #
def init():

    glClearColor(0.1, 0.1, 0.1, 1)

    glMatrixMode(GL_PROJECTION)

    glLoadIdentity()

    gluOrtho2D(-100, 100, -100, 100)

# ===== Main ===== #
def main():

    global shapes

    glutInit()

    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)

    glutInitWindowSize(800, 800)

    glutCreateWindow(b"Painel das Formas")

    init()

```

```
# ===== Formas ===== #

shapes = [

    Square(
        x=-50,
        y=50,
        size=40,
        color=(1, 0, 0)
    ),

    Triangle(
        x=50,
        y=50,
        size=25,
        color=(0, 1, 0)
    ),

    Circle(
        x=-50,
        y=-50,
        radius=20,
        color=(0, 0, 1)
    ),

    Hexagon(
        x=50,
        y=-50,
        radius=20,
        color=(1, 1, 0)
    )
]

glutDisplayFunc(display)

glutMainLoop()

if __name__ == "__main__":
    main()
```

✓ Missão 3: A Aliança dos Triângulos

Desenvolva uma função que desenhe triângulos isósceles com base e altura fornecidas pelo usuário. Em seguida, use essa função para desenhar pelo menos 5 triângulos com diferentes tamanhos e posições em tela. Cada triângulo deve ter uma cor distinta.

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

import random

# ===== Classe Base ===== #
class Shape:

    def __init__(self, x, y, color):

        self.x = x
        self.y = y

        self.color = color

        self.vertices = []
        self.indices = []

# ===== Desenho ===== #
def draw(self):

    glColor3f(*self.color)

    glBegin(GL_TRIANGLES)

    for index in self.indices:

        vx, vy = self.vertices[index]

        glVertex2f(vx + self.x, vy + self.y)
```

```

        glEnd()

# ===== Triângulo Isósceles ===== #
class IsoscelesTriangle(Shape):

    def __init__(self, x, y, base, height, color):

        super().__init__(x, y, color)

        half_base = base / 2
        half_height = height / 2

        # Shared vertex
        self.vertices = [

            (0, half_height),          # topo

            (-half_base, -half_height), # esquerda

            (half_base, -half_height)   # direita

        ]

        # Índices
        self.indices = [
            0, 1, 2
        ]

# ===== Cor Aleatória ===== #
def random_color():

    return (
        random.random(),
        random.random(),
        random.random()
    )

# ===== Lista de Triângulos ===== #
triangles = []

# ===== Display ===== #
def display():

    glClear(GL_COLOR_BUFFER_BIT)

    for triangle in triangles:
        triangle.draw()

    glutSwapBuffers()

# ===== Inicialização ===== #
def init():

    glClearColor(0.05, 0.05, 0.05, 1)

    glMatrixMode(GL_PROJECTION)

    glLoadIdentity()

    gluOrtho2D(-100, 100, -100, 100)

# ===== Criar Triângulo ===== #
def create_triangle():

    base = float(input("Digite a base do triângulo: "))
    height = float(input("Digite a altura do triângulo: "))

    return base, height

# ===== Programa Principal ===== #
def main():

    global triangles

    # Entrada do usuário

```

```

base, height = create_triangle()

glutInit()

glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)

glutInitWindowSize(800, 800)

glutCreateWindow(b"Triangulos Isosceles")

init()

# ===== Cena ===== #

triangles = [

    IsoscelesTriangle(
        x=-70,
        y=60,
        base=base,
        height=height,
        color=random_color()
    ),

    IsoscelesTriangle(
        x=0,
        y=60,
        base=base * 0.7,
        height=height * 1.2,
        color=random_color()
    ),

    IsoscelesTriangle(
        x=70,
        y=50,
        base=base * 1.5,
        height=height * 0.8,
        color=random_color()
    ),

    IsoscelesTriangle(
        x=-40,
        y=-40,
        base=base * 0.5,
        height=height * 0.5,
        color=random_color()
    ),

    IsoscelesTriangle(
        x=50,
        y=-50,
        base=base * 1.2,
        height=height * 1.5,
        color=random_color()
    )
]

glutDisplayFunc(display)

glutMainLoop()

if __name__ == "__main__":
    main()

```

✓ Missão 4: O Domínio do Espaço

Crie uma cena com pelo menos três triângulos posicionados em diferentes quadrantes do plano cartesiano (SRU), e desenhe os eixos X e Y. Exiba um pequeno texto (usando glutBitmapCharacter ou similar) com as coordenadas de cada triângulo.

```

from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

import random

# ===== Classe Triângulo ===== #
class Triangle:

```

```

def __init__(self, x, y, base, height, color):

    self.x = x
    self.y = y

    self.base = base
    self.height = height

    self.color = color

    half_base = base / 2
    half_height = height / 2

    # Shared vertex
    self.vertices = [

        (0, half_height),

        (-half_base, -half_height),

        (half_base, -half_height)
    ]

    self.indices = [
        0, 1, 2
    ]

# ===== Desenho ===== #
def draw(self):

    glColor3f(*self.color)

    glBegin(GL_TRIANGLES)

    for index in self.indices:

        vx, vy = self.vertices[index]

        glVertex2f(vx + self.x, vy + self.y)

    glEnd()

# ===== Coordenadas ===== #
def get_position_text(self):

    return f"({self.x}, {self.y})"

# ===== Lista de triângulos ===== #
triangles = []

# ===== Desenhando Texto ===== #
def draw_text(text, x, y):

    glColor3f(1, 1, 1)

    glRasterPos2f(x, y)

    for char in text:

        glutBitmapCharacter(
            GLUT_BITMAP_HELVETICA_18,
            ord(char)
        )

# ===== Desenhando Eixos ===== #
def draw_axes():

    glLineWidth(2)

    glColor3f(1, 1, 1)

    glBegin(GL_LINES)

    # Eixo X
    glVertex2f(-100, 0)
    glVertex2f(100, 0)

    # Eixo Y

```



```

        glVertex2f(0, -100)
        glVertex2f(0, 100)

    glEnd()

# ===== Display ===== #
def display():

    glClear(GL_COLOR_BUFFER_BIT)

    # Desenha eixos
    draw_axes()

    # Desenha triângulos
    for triangle in triangles:

        triangle.draw()

    # Texto das coordenadas
    draw_text(
        triangle.get_position_text(),
        triangle.x - 15,
        triangle.y - 25
    )

    glutSwapBuffers()

# ===== Inicialização ===== #
def init():

    glClearColor(0.08, 0.08, 0.08, 1)

    glMatrixMode(GL_PROJECTION)

    glLoadIdentity()

    gluOrtho2D(-100, 100, -100, 100)

# ===== Main ===== #
def main():

    global triangles

    glutInit()

    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)

    glutInitWindowSize(800, 800)

    glutCreateWindow(b"Plano Cartesiano - Triangulos")

    init()

# ===== Triângulos ===== #

    triangles = [

        # Quadrante I
        Triangle(
            x=50,
            y=50,
            base=20,
            height=30,
            color=(1, 0, 0)
        ),

        # Quadrante II
        Triangle(
            x=-50,
            y=40,
            base=25,
            height=35,
            color=(0, 1, 0)
        ),

        # Quadrante III
        Triangle(
            x=-40,
            y=-50,

```

```

        base=30,
        height=25,
        color=(0, 0, 1)
    ),

    # Quadrante IV
    Triangle(
        x=50,
        y=-20,
        base=30,
        height=25,
        color=(1, 0, 1)
    )
]

glutDisplayFunc(display)

glutMainLoop()

if __name__ == "__main__":
    main()

```

✓ Missão 5: A Insignia de Malta

Reconstruir um antigo símbolo conhecido como "Insignia de Malta", usado para desbloquear o "Portal dos Pixels". Para isso, crie uma função chamada `desenharInsignia()` que monte uma figura simétrica utilizando apenas retângulos e triângulos. Ao pressionar a tecla "C" deve alterar as cores do símbolo aleatoriamente (como se ele ganhasse "energia mágica").

```

from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
import math
import random

# ===== Classe Base ===== #
class Shape:

    def __init__(self, vertices, color):
        self.vertices = vertices
        self.indices = [0, 1, 2]
        self.color = color

    # ===== Desenhar ===== #
    def draw(self):
        glColor3f(*self.color)
        glBegin(GL_TRIANGLES)

        for index in self.indices:
            vx, vy = self.vertices[index]
            glVertex2f(vx, vy)

        glEnd()

    # ===== Nova Cor ===== #
    def randomize_color(self):

        self.color = (
            random.random(),
            random.random(),
            random.random()
        )

# ===== Triângulo ===== #
class Triangle(Shape):

    def __init__(self, vertices, color):

        super().__init__(vertices, color)

        self.indices = [0, 1, 2]

# ===== Insignia de Malta ===== #
class MaltaInsignia:

```

```

def __init__(self):
    self.triangles = []
    self.create_insignia()

# ===== Construção ===== #
def create_insignia(self):
    outer_radius = 90 # distância do centro às pontas externas
    notch_radius = 60 # distância do centro ao entalhe em V
    half_angle = 30 # meia largura angular de cada braço (graus)

    color = self.random_color()
    center = (0.0, 0.0)

    # 4 braços: cima, direita, baixo, esquerda
    arm_directions = [90, 0, -90, 180]

    for arm_dir in arm_directions:
        a_left = math.radians(arm_dir + half_angle)
        a_right = math.radians(arm_dir - half_angle)
        a_mid = math.radians(arm_dir)

        tip_left = (outer_radius * math.cos(a_left), outer_radius * math.sin(a_left))
        tip_right = (outer_radius * math.cos(a_right), outer_radius * math.sin(a_right))
        notch = (notch_radius * math.cos(a_mid), notch_radius * math.sin(a_mid))

        # Dois triângulos por braço formando o V externo
        self.triangles.append(Shape([center, tip_left, notch], color))
        self.triangles.append(Shape([center, notch, tip_right], color))

# ===== Cor Aleatória ===== #
def random_color(self):

    return (
        random.random(),
        random.random(),
        random.random()
    )

# ===== Desenhar ===== #
def draw(self):
    for tri in self.triangles:
        tri.draw()

# ===== Atualizar cores ===== #
def energize(self):
    for tri in self.triangles:
        tri.randomize_color()

# ===== Objeto Global ===== #
insignia = MaltaInsignia()

# ===== Display ===== #
def display():
    glClear(GL_COLOR_BUFFER_BIT)
    insignia.draw()
    glutSwapBuffers()

# ===== Teclado ===== #
def keyboard(key, x, y):
    # Pressione C
    if key == b'c' or key == b'C':
        insignia.energize()
        glutPostRedisplay()

# ===== Inicialização ===== #
def init():
    glClearColor(0.05, 0.05, 0.08, 1)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluOrtho2D(-100, 100, -100, 100)

# ===== Main ===== #
def main():
    glutInit()
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)
    glutInitWindowSize(800, 800)
    glutCreateWindow(b"Insignia de Malta")
    init()
    glutDisplayFunc(display)
    glutKeyboardFunc(keyboard)
    glutMainLoop()

```

```
if __name__ == "__main__":
    main()
```

✓ Missão 6: A Constelação dos Guardiões

Construa a “Constelação dos Guardiões”, formada por um conjunto de círculos de diferentes tamanhos, que representam estrelas mágicas conectadas por propósito. Crie uma função `desenharEstrela(x, y, raio)` para desenhar um círculo em uma posição específica. Utilize essa função para montar uma constelação composta por pelo menos 7 estrelas, cada uma com:

- Um raio diferente (variação entre 10 e 50) e cores aleatórias gerada a cada execução.
- Conexões entre estrelas feitas por linhas finas (use `GL_LINES`).
- Recursos interativos:
 - Pressionar a tecla "n" para adicionar uma nova estrela com raio e posição aleatórios.
 - Pressionar a tecla "x" para remover uma das estrelas (pode ser aleatoriamente ou não).
 - Pressionar a tecla "r" reinicia a constelação.
 - Pressionar "t" ativa o “modo noturno” (plano de fundo preto e estrelas brilhantes em branco ou amarelo).

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

import random
import math

# ===== CONFIGURAÇÕES ===== #

WINDOW_WIDTH = 900
WINDOW_HEIGHT = 900

MIN_RADIUS = 10
MAX_RADIUS = 50

MIN_COORD = -90
MAX_COORD = 90

INITIAL_STARS = 7

# ===== CLASSE ESTRELA ===== #
class Star:
    def __init__(self, x, y, radius, color):

        self.x = x
        self.y = y

        self.radius = radius

        self.color = color

        self.vertices = []
        self.indices = []

        self.generate_circle()

# ===== GERAÇÃO DO CÍRCULO ===== #

def generate_circle(self, segments=40):

    # vértice central
    self.vertices.append((0, 0))

    # borda do círculo
    for i in range(segments):

        angle = 2 * math.pi * i / segments

        vx = self.radius * math.cos(angle)
        vy = self.radius * math.sin(angle)

        self.vertices.append((vx, vy))

    # triangulação
    for i in range(1, segments):
```

```

        self.indices.extend([
            0, i, i + 1
        ])

    self.indices.extend([
        0, segments, 1
    ])

def draw(self):
    glColor3f(*self.color)
    glBegin(GL_TRIANGLES)

    for index in self.indices:
        vx, vy = self.vertices[index]
        glVertex2f(vx + self.x, vy + self.y)
    glEnd()

# ===== CONSTELAÇÃO ===== #
class Constellation:
    def __init__(self):
        self.stars = []
        self.night_mode = False
        self.generate_initial_stars()

    # COR ALEATÓRIA
    def random_color(self):

        return (
            random.random(),
            random.random(),
            random.random()
        )

    # MODO NOTURNA
    def night_color(self):

        colors = [

            (1, 1, 1),      # branco

            (1, 1, 0.6),    # amarelo claro

            (1, 0.9, 0.5)

        ]

        return random.choice(colors)

    # ===== CRIAÇÃO DAS ESTRELAS ===== #
    def create_star(self):

        x = random.randint(MIN_COORD, MAX_COORD)

        y = random.randint(MIN_COORD, MAX_COORD)

        radius = random.randint(MIN_RADIUS, MAX_RADIUS)

        if self.night_mode:
            color = self.night_color()
        else:
            color = self.random_color()

        return Star(x, y, radius, color)

    # ===== GERAÇÃO INICIAL ===== #

    def generate_initial_stars(self):

        self.stars.clear()

        for _ in range(INITIAL_STARS):

            self.stars.append(
                self.create_star()
            )

    # ADICIONAR ESTRELA
    def add_star(self):
        self.stars.append(
            self.create_star()
        )

    # REMOVER ESTRELA

```

```

# REMOVER ESTRELA
def remove_star(self):
    if len(self.stars) > 0:
        index = random.randint(
            0,
            len(self.stars) - 1
        )
        self.stars.pop(index)

# RESETAR
def reset(self):
    self.generate_initial_stars()

# MODO NOTURNO
def toggle_night_mode(self):
    self.night_mode = not self.night_mode

    for star in self.stars:
        if self.night_mode:
            star.color = self.night_color()
        else:
            star.color = self.random_color()

# CONEXÕES
def draw_connections(self):

    glLineWidth(1.5)

    if self.night_mode:
        glColor3f(0.6, 0.6, 1)
    else:
        glColor3f(1, 1, 1)

    glBegin(GL_LINES)

    for i in range(len(self.stars) - 1):

        s1 = self.stars[i]
        s2 = self.stars[i + 1]

        glVertex2f(s1.x, s1.y)
        glVertex2f(s2.x, s2.y)

    glEnd()

# DESENHAR CONSTELAÇÃO
def draw(self):
    self.draw_connections()

    for star in self.stars:
        star.draw()

constellation = Constellation()

# ===== DISPLAY ===== #
def display():
    if constellation.night_mode:
        glClearColor(0, 0, 0, 1)

    else:
        glClearColor(0.08, 0.08, 0.12, 1)

    glClear(GL_COLOR_BUFFER_BIT)
    constellation.draw()
    glutSwapBuffers()

# ===== TECLADO ===== #
def keyboard(key, x, y):
    # adicionar estrela
    if key == b'n':
        constellation.add_star()
    # remover estrela
    elif key == b'x':
        constellation.remove_star()
    # resetar
    elif key == b'r':
        constellation.reset()
    # modo noturno
    elif key == b't':
        constellation.toggle_night_mode()
    glutPostRedisplay()

```

```
# ===== INICIALIZAÇÃO ===== #
def init():

    glMatrixMode(GL_PROJECTION)

    glLoadIdentity()

    gluOrtho2D(-100, 100, -100, 100)

# ===== MAIN ===== #
def main():

    glutInit()

    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)

    glutInitWindowSize(
        WINDOW_WIDTH,
        WINDOW_HEIGHT
    )

    glutCreateWindow(
        b"Constelacao dos Guardioes"
    )

    init()

    glutDisplayFunc(display)

    glutKeyboardFunc(keyboard)

    glutMainLoop()

if __name__ == "__main__":
    main()
```